# BD-theory2-resumen.pdf

biingus

Bases de Datos

2º Grado en Ingeniería Informática del Software

Escuela de Ingeniería Informática
Universidad de Oviedo

## FUNCTIONAL DEPENDENCIES

Customer (DNI, name, city, PC, province)

$F_0$:
- PC → province (postal code implies a specific province)
- province → PC
- DNI → name
- DNI → city
- DNI → name, city, PC, province

$F_1$: PC → province; DNI → name, city, PC, province

We can delete province because DNI → PC and PC → province, so DNI → province is unnecessary. TRANSITIVITY

$F_2$: PC → province; DNI → name, city, PC

MINIMAL COVER → $F_0 \equiv F_1 \equiv F_2$

**MINIMAL COVER ($F_c$)**

- Set that is equivalent to other, without redundant dependencies.

EXTRANEOUS ATTRIBUTE: attributes that can be deleted and the set is still equivalent.

## ARMSTRONG RULES

1. REFLEXIVITY: A → B  B ⊆ A
2. AUGMENTATION: A → B  wA → wB
3. TRANSITIVITY: A → B, B → C  A → C
4. DECOMPOSITION: A → BC  A → B, A → C
5. UNION: A → B, A → C  A → BC
6. PSEUDOTRANSITIVITY: A → B, BW → C  AW → C

## KEYS

SUPERKEY (K): K → R, Any set of attributes that can IDENTIFY an entity.

DNI → PC → Province; DNI → name; DNI → city

CLOSURE OF SET OF ATTRIBUTES (FIND KEYS)

R = (A, B, C, D, E, F, G, H)

F = { AB → C; C → DE; E → HF }

# CLOSURES OF L-SIDE:

(AB)⁺ = {A, B, C, D, E, H, F}  ABG → ABCDEFGH → R

## REDUNDANCIES

Customer (DNI, name, city, PC, province)

1 Paco OVD 33009 AST

1 Antonio → DNI = 1 is associated with Paco

2 Carmen OVD 33009 LEON → 33009 is associated with AST

Forced to always repeat the same values → REDUNDANCY

# If left side can appear multiple times → ONLY KEYS IN LEFT SIDE

F = { DNI → name, city, PC; PC → province }

PC is not part of a key: NEW TABLE where PC is key:

Customer (DNI, name, city, PC) — DECOMPOSITION

Postal_Codes (PC, province)

## DECOMPOSITION

- Divide a table into smaller ones to avoid REDUNDANCIES
- $r(R)$  $r_1(R_1)$ $r_2(R_2)$ → $r = r_1 \bowtie r_2 \bowtie \dots \bowtie r_n$

LOSSLESS JOIN: if you join the derived tables you get the original one.

$R_1$ = (DNI, name, city, PC)  $R_2$ = (PC, province)

$F_1$ = { DNI → name city PC province }  $F_2$ = { PC → province }

# JOINING ATTRIBUTES SHOULD BE KEYS TO AVOID LOSSY JOINS

It needs to be stated because it not longer appears in the table

## NORMAL FORMS

- The higher the NF, less redundancy

BCNF (Boyce-Codd NF)

1. X → Y is trivial
2. X → is a SUPERKEY

Left sides MUST be SUPERKEYS

# STEPS

R = (A, B, C, D, E, F, G, H)

F = { AB → C; C → DE; E → FH }  KEY: ABG; C, E NOT KEYS

$R_1$ = (E, F, G)
$R_2$ = (C, D, E)
$R_3$ = (A, B, C)
$R_4$ = (A, B, G)

R = (Customer, Branch, Advisor)

F = { Customer Branch → Advisor; Advisor → Branch }

C is ISOLATED: WILL BE PART OF ALL KEYS

KEYS: CA, CB

F = { CB → A; xA → B }

NOT A SUPERKEY! (only part of one)

$R_1$ = (A, B)  $F_1$ = { A → B }

$R_2$ = (C, A)  $F_2$ = { } — No dependencies → PK

## DEPENDENCY PRESERVING

R = (A, B, C)

F = { A → B; B → C; C → A }

$R_1$ = (A, B)  $R_2$ = (B, C)

$F_1$ = { A → B, B → A }  $F_2$ = { B → C, C → B }

Not originally in F

By union ($F_1 \cup F_2$) you get C → A

C → B ∪ B → A = C → A

## 3NF

# Decompositions are not DEPENDENCY PRESERVED

PROPERTIES

1. X → Y is trivial
2. X is SUPERKEY
3. All Y are PRIME → PART OF KEY

R = (customer, branch, advisor, office)

F = { CB → A; A → BO }

$R_1$ = (C, B, A)  $F_1$ = { CB → A, A → B }

$R_2$ = (A, B, O)  $F_2$ = { A → BO }

## ALGORITHM

0. F should be MINIMAL COVER
1. Each dependency gets a TABLE
2. Delete (if possible) repeated dependencies.
3. If no candidate keys of original decomposition CREATE TABLE with those attributes

## EXERCISE

R = (A, B, ..., I) → G

F = { ABC → DE; E → HF; H → IBC }

# NO INCOMING ARROWS → PART OF KEY: A, G

# KEYS: BCAG, HAG, EAG

1. TABLE PER RELATION
2. REPEATED DEPENDENCIES
3. CANDIDATE KEY IN F? G IS MISSING

3NF  $R_1$ = (A, B, C, D, E)  $F_1$ = { ABC → DE, E → BC }

BCNF  $R_2$ = (E, F, H)  $F_2$ = { E → HF }

BCNF  $R_3$ = (H, I, B, C)  $F_3$ = { H → IBC }

BCNF (!) $R_4$ = (A, B, G)  $F_4$ = { } CANDIDATE KEY: NO DEPENDENCIES